

10. Übungsblatt zur Vorlesung Fortgeschrittene Algorithmen (WS 2025/26)

Aufgabe 1 – Platzeffiziente Baumstrukturen (LOUDS)

In der Vorlesung haben wir *Succinct Data Structures* kennengelernt, die es ermöglichen, Daten nahe am informationstheoretischen Minimum zu speichern. Ziel dieser Aufgabe ist der Vergleich einer klassischen, zeigerbasierten Implementierung eines Baumes mit dessen LOUDS-Repräsentation (*Level-Order Unary Degree Sequence*) in C++.

Mit der Datei `BitVector.hpp` ist eine grundlegende Implementierung eines Bitvektors mit den Operationen `rank` und `select` gegeben. Diese Datei dient als Bibliothek und sollte nicht verändert werden. Der zu implementierende Code gehört in die Datei `main.cpp`, in der bereits ein Skelett zur Orientierung gegeben ist.

a) Implementieren Sie zunächst in der Klasse `BaselineTree` (in `main.cpp`) eine traditionelle Datenstruktur für allgemeine Bäume als Baseline. Bei der Darstellung des Graphen können sie frei aus Adjazenzliste sowie -Matrix, Mengen und Zeigern wählen. Wichtig ist nur, dass die Struktur dynamisch wachsen kann.

- Entwerfen Sie die traditionelle Baum-Speicherstruktur.
- Implementieren Sie eine Methode `sizeInBytes()`, die den tatsächlichen Speicherverbrauch des gesamten Baumes in der Baseline-Struktur berechnet. Berücksichtigen Sie dabei den Overhead (also unter anderem `sizeof(Node)`, den Overhead von Containern, die Größe der Zeiger etc.).
- Implementieren Sie in der `main`-Funktion eine Methode, die einen zufälligen Baum mit N Knoten in Ihrer Baseline-Struktur erzeugt.

6 Punkte

b) Implementieren Sie die Klasse `LOUDSTree`. Es ist nicht vorgesehen, Knoten direkt in den LOUDS-Baum einzufügen. Stattdessen wird der Baum einmalig aus der Baseline-Datenstruktur konstruiert und bleibt anschließend statisch. Schreiben Sie einen Konstruktor, der eine Instanz von `BaselineTree` entgegennimmt. Traversieren Sie diesen Baum und generieren Sie den entsprechenden LOUDS-Bitstring.

Die Klasse `LOUDSTree` darf keine Zeiger oder Knoten-Objekte speichern, sondern nur den `BitVector`. Beachten Sie die Konvention, dass der Bitstring mit der Sequenz 10 (für die Super-Wurzel) beginnt. Rufen Sie am Ende zwingend `bv.buildIndex()` auf.

4 Punkte

c) Implementieren Sie für den LOUDS-Baum anschließend die folgenden Methoden ausschließlich durch Aufrufe von `rank0/1` und `select0/1`:

- `uint64_t parent(uint64_t i)`: der Elternknoten des Knoten `i`, falls vorhanden
- `bool isRoot(uint64_t i)`: 1 falls `i` die Wurzel ist, 0 sonst
- `bool isLeaf(uint64_t i)`: 1 falls `i` ein Blatt ist, 0 sonst
- `uint64_t outDegree(uint64_t i)`: der ausgehende Grad von `i` (also die Anzahl der Kinder von `i`)
- `uint64_t childNum(uint64_t i, uint64_t j)`: der Index des `j`-ten Kindes von `i`, falls es existiert, und 0 sonst

6 Punkte

d) Führen Sie mit dem vorgegebenen Benchmark-Code in der `main.cpp` eine Messung mit $N = 1.000.000$ Knoten durch. Diskutieren Sie anschließend den tatsächlichen Speicherplatzbedarf in Byte der traditionellen sowie der LOUDS-Implementierung des gleichen Baumes. Liegt das Ergebnis nahe an der theoretischen Erwartung von ≈ 2 Bits pro Knoten?

Betrachten Sie außerdem die Laufzeitkomplexität der von Ihnen implementieren Funktionen. Nehmen Sie zu diesem Zweck an, dass `rank` und `select` in $O(1)$ laufen. Wie groß wäre der Aufwand, auf der LOUDS-Datenstruktur dynamisch zu arbeiten, d.h. Knoten neu anlegen und löschen zu können?

4 Punkte

Bitte geben Sie Ihre Lösungen bis **Mittwoch, 28. Januar 2026, 12:00 Uhr** einmal pro Gruppe über Moodle als pdf-Datei ab. Vermerken Sie dabei stets die Namen aller Bearbeitenden auf der Abgabe.

Grundsätzlich sind stets alle Ihrer Aussagen zu begründen und Ihr Pseudocode ist stets zu kommentieren.